

1. Project Overview & Scope

Koki Woki ChefBot is an advanced conversational AI cooking assistant designed to deliver intelligent recipe discovery, user preference modeling, and proactive allergy/dietary tracking. The system features a highly-decoupled architecture composed of a desktop Graphic User Interface (GUI) and a standard Console CLI, demonstrating best practices in modular software engineering.

2. Technical Project Capabilities

- **Explainable AI (XAI) Recommendation:** Utilizes user preference weights to score recipe entries, combined with an explainability module that outputs plain text rationales (e.g., 'Since you prefer Italian, you might enjoy...')
- **Dual-Interface Presentation:** Supports a premium event-driven Desktop GUI utilizing custom graphic rendering, FlatLaf Dark styling, and thread-safe viewport scrolling, alongside a fallback interactive Console shell.
- **Multi-State Conversational State Machine:** Tracks user conversation steps and guides them step-by-step through interactive recipes, caching step numbers and dynamically validating command arguments.
- **Boolean Multi-Term Search & Regex Parsing:** Decodes compound queries (e.g., 'chicken and pasta') and performs strict set intersection indexing, combined with Regex parsers to capture preparation times.
- **Positive/Negative Preference Constraints:** Detects negative phrases (e.g., 'dont like', 'avoid') to populate disfavored lists, proactively filtering these out and serving custom warn-dialogs if a match occurs.

3. Source Modularity & File Mapping

The codebase consists of 10 clean Scala classes, organized hierarchically to maintain structural decoupling. Each module operates in a dedicated package namespace.

Filename	Package Layer	Primary System Responsibility
KokiWokiGUI.scala	chatBot.gui	Desktop GUI. Handles layouts, FlatLaf styles, rounded message bubble canvas painting, and thread-safe scrolling.
ChatController.scala	chatBot.controller	Mediates GUI clicks and text inputs with database layers and tracks current logged-in sessions.
CookingChatBot.scala	(top-level)	Entry point for the CLI. Drives command loop, terminal auth prompts, and session history restores.
ChatBotInteract.scala	chatBot.ui	Central chatbot router ('Chatbot'). Controls dynamic dialog state flows and routes inputs to specialized handlers.
ConversationBrain.scala	chatBot.brain	Natural Language Processing (NLP) layer. Smart ranked keyword search, regex parsing, and mood/topic calculations.
Recipes.scala	chatBot.data	Houses structural database schemas ('Recipe' and 'InteractionEntry') and the 46-recipe dataset.
Recommendations.scala	chatBot.recommend	Applies dynamic progressive relaxation algorithms to recommend recipes and generates explainability text.
Preferences.scala	chatBot.engine	Disk-backed configuration database tracking preferred and avoided ingredients/cuisines per user.
Login.scala	chatBot.auth	Authenticates login credentials, manages accounts, and serializes historical chat transcripts.
FormattedResponse.scala	chatBot.response	Stateless, pure string layouts formatting cards, summaries, and guides.

4. Architectural Modularity & Mediator Pattern

To support both Graphical and Console interactions without duplicating code, Koki Woki ChefBot implements a clean ****Model-View-Controller (MVC) / Mediator**** architectural pattern. The system is layered vertically with no circular dependencies.

Architectural Layout Hierarchy

- **Layer 1: Presentation Layer:** Handles all input/output renderings. Includes ModernKokiWokiUI (Scala Swing Desktop View) and startChefBot (Terminal Interactive Console View).
- **Layer 2: Mediation Layer:** Represented by ChatController.scala. It abstracts all session tracking, authentication handshakes, and preference state updates, shielding the Swing view from directly referencing core logical engines.
- **Layer 3: Core Application Router:** Represented by ChatBotInteract.scala. Orchestrates the flow of queries, matches intent states, and updates dynamic active step-by-step guides.
- **Layer 4: Business Services Layer:** Consists of specialized, stateless utility components: UserAuth (identity operations), PreferenceManager (likes/dislikes persistence), RecommendationEngine (taste recommendations), and ResponseFormatter (pretty prints).
- **Layer 5: Analytical Intelligence Layer:** Consists of ConversationBrain.scala. Houses NLP calculation utilities: weighted search indices, regex extractors, and emotional classifiers.

Advantages of the Mediator Architecture

1. **Zero Presentation Coupling:** The GUI code is purely graphical and contains zero chatbot brain logic, which is completely mediated by the Controller.
2. **Reusable Logical Core:** The exact same `Chatbot` router and brain files are shared between the CLI and GUI, ensuring 100% consistent feature matching.
3. **Multi-session Consistency:** The Controller reads preferences directly from the file database on every call, avoiding stale state caches.

5. Dynamic Conversational State Machine

Traditional chatbots rely on flat, single-turn search. Koki Woki ChefBot employs an encapsulated, conversational State Machine to drive interactive step-by-step processes and conversational branches.

1. Step-by-Step Interactive Cooking Guide

When a user asks to cook a recipe (e.g., 'start cooking lasagna'), the state machine triggers a cooking pipeline:

- The current recipe schema is cached in the ``cookingRecipe: Option[Recipe]`` state parameter.
- The step pointer ``cookingStep`` is initialized to 0.
- The chatbot displays the first instruction, transitioning to the active cooking state.
- The main router intercepts subsequent inputs. When the user types 'done', the pointer increments and the next step is dynamically formatted and returned. Typing done on the final step resets all state pointers and celebrates completion.

2. Context-Aware Sub-Dialog Flows

To solve the 'cold-start' recipe discovery issue, the state machine tracks branching paths (e.g., ``quick_meal``, ``comfort_food``, ``healthy_food``) via an encapsulated ``currentFlow: Option[String]`` variable:

- **State Transition:** Typing 'quick meal' transitions ``currentFlow`` to ``Some("quick_meal")`` and asks 'How much time do you have? 10, 20, or 30+ minutes?'.
- **Branch Execution:** The subsequent user entry (e.g., '10') is routed specifically under the active flow context, fetching matching recipes immediately and then resetting the flow state to ``None``.

6. Semantic Search & Negative Preferences

The system features a weighted relevance indexing engine combined with smart exclusion filtering, protecting users from allergies and food dislikes.

1. Weighted Multi-Attribute Relevance Index

When search queries are submitted, keywords are tokenized and scored across multiple data layers:

- **Recipe Name Match:** +10 points
- **Cuisine Match:** +15 points
- **Ingredient Match:** +5 points
- **Dietary Tag Match:** +8 points

Results are sorted descending, returning only highly relevant, targeted suggestions.

2. Proactive Negative Constraint Filtering

Users can specify items they wish to avoid (e.g., 'I hate spicy food' or 'no chicken'). The PreferenceManager extracts these parameters and saves them under the `avoid` key. The engines then execute exclusion filtering:

- **Query Filtering:** During searches and recommendation computations, recipes containing disfavored cuisines or tags are filtered out completely.
- **Intelligent Warning:** If a high-scoring recipe matches the query term but contains an avoided element, it is omitted, and the chatbot displays a customized dialog: *'I found Recipe X, but it contains something you are avoiding (Spicy). Would you like me to recommend something else?'*

7. Scala 3 Collection API & Monadic Index

The codebase stands as a model for pure Functional Programming, leveraging monadic collections and higher-order functions to execute branchless, safe calculations.

Function	Applied Module	Functional Software Purpose
<code>.map(f)</code>	All modules	Transforms elements of Lists or monadic Options cleanly without side-effects.
<code>.filter(pred)</code>	Chatbot / Search	Keeps only elements satisfying a predicate; excludes disfavored items.
<code>.flatMap(f)</code>	ConversationBrain	Maps then flattens nested lists (e.g., aggregating distinct ingredients across recipes).
<code>.find(pred)</code>	Chatbot Router	Returns <code>Option[T]</code> containing the first matching element, preventing null failures.
<code>.exists(pred)</code>	ConversationBrain	Tests boolean criteria across a list (e.g., looking for disallowed items).
<code>.forall(pred)</code>	RecommendationEngine	Vacuously true when an Option is None; acts as a branchless safe filter.
<code>.count(pred)</code>	ConversationBrain	Counts matching elements (e.g., tracking emotional keywords in chat histories).
<code>.distinct</code>	ConversationBrain	Deduplicates autocomplete suggestion lists and aggregated topics.
<code>.sortBy(f)</code>	ConversationBrain	Sorts search results in descending order based on weighted score integers.
<code>.zipWithIndex</code>	KokiWokiGUI	Pairs items with indexes to render numbered instructions dynamically.
<code>Random.shuffle</code>	Recommendations	Randomly shuffles recommendations to provide fresh suggestions.

8. Java AWT & Scala Swing API Index

The presentation view layer harnesses the full capability of the Swing layout framework combined with custom anti-aliased graphics painting to produce a responsive desktop client.

Graphical API	Applied Module	UI & Layout Responsibility
SimpleSwingApplication	KokiWokiGUI.scala	Manages the Swing application life cycle, window startups, and frame rendering.
MainFrame	KokiWokiGUI.scala	Renders the primary desktop application frame and window metadata.
BoxPanel / BorderPanel	KokiWokiGUI.scala	Organizes sub-panels vertically, horizontally, or across cardinal boundaries.
ScrollPane	KokiWokiGUI.scala	Wraps chat containers to provide fluid, automatic scrolling viewports.
Graphics2D	KokiWokiGUI.scala	Provides advanced anti-aliased custom painting routines.
GradientPaint	KokiWokiGUI.scala	Generates smooth, professional background color gradients for glassmorphism panels.
RenderingHints	KokiWokiGUI.scala	Configures graphics rendering pipeline for beautiful, rounded edge bubbles.
SwingUtilities.invokeLater	KokiWokiGUI.scala	Safely schedules scroll updates on the Swing Event Dispatch Thread (EDT).
JOptionPane	KokiWokiGUI.scala	Displays native dialog popups for registration prompts and password inputs.
FlatDarkLaf	KokiWokiGUI.scala	Sets up the flatdark-skinned Look-and-Feel theme interface.

9. End-to-End System Data Flow

The system handles user inputs through a unified processing pipeline, converting raw textual and graphical events into context-aware responses and recommendations.

Step 1: Event Dispatch: User submits text via GUI TextField or Console StdIn. readline. GUI events trigger actionPerformed listeners, dispatching inputs to the Controller.

Step 2: Controller Mediation: ChatController intercepts the message, checks user auth state, and dispatches the payload to the Chatbot router.

Step 3: Intent & Context Extraction: Chatbot normalizes inputs to lowercase. It calls ConversationBrain to check mood, parse cuisines, extract tags, and detect prep-times via Regex.

Step 4: Preference Constraint Merge: If preference keywords (e.g. 'I hate') are matched, PreferenceManager stores dislikes/likes in local configuration files on disk.

Step 5: Proactive Exclusion Filtering: Calculates searches and recommendations, automatically omitting recipes matching disfavored ingredients or tags, serving warnings if a direct match occurs.

Step 6: Dialog State Routing: Evaluates conversational flow variables (Option[String]). Routes queries to specialized flows or the step-by-step cooking engine.

Step 7: Stateless Formatting: Pipes recipe lists and guide strings to the ResponseFormatter to pretty-print structured visual cards.

Step 8: Thread-Safe UI Update: The UI receives the formatted text. In the GUI, SwingUtilities.invokeLater is called to recalculate scroll heights and scroll the chat view to the bottom.

10. Structural Software Design Patterns

Koki Woki ChefBot implements standard structural and behavioral software engineering patterns to ensure high maintenance, performance, and decoupling:

■ Model-View-Controller (MVC) / Mediator:

Decouples View containers (KokiWokiGUI) from database services. ChatController acts as a mediator, abstracting logins, preference retrievals, and engine dispatches.

■ Singleton Object Pattern:

Scala's object keyword creates JVM-safe single shared instances for all services (ConversationBrain, RecommendationEngine, PreferenceManager, ResponseFormatter), saving memory.

■ State Machine Pattern:

Conversational loops and cooking guidelines represent dynamic states managed by Option[String] parameters, allowing branching conversations.

■ Template Method / Graphics Painting Hooks:

Overrides paintComponent(g: Graphics2D) to draw rounded bubble chat corners and custom background gradients, providing a beautiful custom UI.

■ Progressive Relaxation Pattern:

RecommendationEngine tries the most specific filter combination first (cuisine + tag + difficulty) and progressively drops criteria if no recipes are found, ensuring the user always receives recommendations.

End of Report — Koki Woki ChefBot Technical Reference